

CyberSentinel AI

Technical Documentation

Threat-Informed Autonomous Security Testing

Smart India Hackathon 2026 · Problem Statement SIH26153

Cowrie SSH Honeypot → MITRE ATT&CK TTP Extraction → Multi-Agent CVE/Config Scanner → LSTM World-Model Attack
Forecasting

Team: Utkarsh Chaubey · Ayush Verma · Umang Srivastava · Mohd Uvais

Document prepared September 2026

Contents

- 1. Executive Summary 4
- 2. Problem Statement Context 5
- 3. System Architecture 6
- 4. Technology Stack 7
- 5. Methodology & Process 9
 - 5.1 Honeypot Capture & Attack Simulation 9
 - 5.2 Log-to-Feature Pipeline 9
 - 5.3 TTP Extraction & MITRE ATT&CK Mapping 9
 - 5.4 Multi-Agent CVE & Configuration Scanner 10
 - 5.5 AI World Model — Design & Training 10
 - 5.6 Explainability 11
 - 5.7 Live Dashboard & Offline Demo Interface 11
- 6. Achievements & Verified Results 12
 - 6.1 Real packet-level capture, not simulated constants 12
 - 6.2 Technique-classification head trained and accurate 12
 - 6.3 Genuine multi-step forecasting 12
 - 6.4 Working offline demo interface 12
 - 6.5 Honest baseline benchmarking 12
- 7. PS26153 Compliance Summary 13
- 8. Known Limitations & Honest Caveats 14
 - 8.1 World Model does not yet beat the baseline on binary F1 14
 - 8.2 Twelve of thirty features have degenerate normalisation ranges 14
 - 8.3 A real data-loss regression was found and fixed mid-project 14
 - 8.4 Three technique-name registries are not yet unified 14
 - 8.5 Single-machine deployment 14
- 9. Roadmap to a Production Version 15
 - 9.1 Close the benchmark gap (highest priority, lowest cost) 15
 - 9.2 Multi-sensor, multi-honeypot data pipeline 15
 - 9.3 Model lifecycle & MLOps 15

9.4 Production-grade serving & security operations integration	15
9.5 Hardening & compliance	15
10. Estimated Costs for a Production Build	17
10.1 One-time engineering effort	17
10.2 Recurring infrastructure & service costs	17
10.3 What is already free / zero marginal cost	18
11. Development & Verification Process	19
12. Conclusion	20

1. Executive Summary

CyberSentinel AI is an autonomous, threat-informed security testing platform built for Smart India Hackathon problem statement SIH26153. It lures real attacker behaviour into an isolated Cowrie SSH honeypot, converts the raw session logs into MITRE ATT&CK-mapped technique records, runs a three-agent AI pipeline (recon, real CVE lookup, configuration audit) to turn those techniques into a ranked, explainable vulnerability report, and — as the project's core AI contribution — trains an LSTM-with-attention **"World Model"** that learns the temporal dynamics of an attack: given the last five observed network states, it predicts (a) the probability the session is a real compromise, (b) which MITRE technique is most likely to come next, and (c) a multi-step forecast of what the network state itself will look like several steps ahead — by recursively feeding its own predictions back in as input.

This document is an honest, as-built technical record of the system: what technologies were used, how each stage of the pipeline actually works, what has been verified end-to-end on real captured traffic, what still falls short of the problem statement's requirements, and what it would take — technically and financially — to carry this from a hackathon prototype to a production-grade deployment. Figures reported as "verified" in this document were produced by real training/benchmark runs on the development machine, not projected or assumed values.

30	Real network features per flow	26	MITRE technique detection rules	3	Parallel AI agents (recon/CVE/config)	5→1	Timestep window → forecast horizon
----	--------------------------------	----	---------------------------------	---	---------------------------------------	-----	------------------------------------

Bottom line: 5 of the 6 explicit SIH26153 requirements are genuinely and verifiably met (see Section 9). The one open gap is that the World Model's binary infiltration-detection F1 score does not yet exceed a simple logistic-regression baseline on the current dataset — a real, measured result, reported honestly rather than rounded up. Section 10 explains why, and Section 11 lays out the two concrete paths to closing it.

2. Problem Statement Context

SIH26153 — Threat-Informed Autonomous Security Testing. The problem statement calls for a system that moves security testing away from static, manually-scheduled scans and toward a pipeline that: (1) observes real attacker behaviour rather than only simulated signatures, (2) maps that behaviour onto a recognised adversary-technique framework (MITRE ATT&CK), (3) automatically correlates observed techniques with real, current CVE data and target-side misconfigurations, (4) builds network-traffic features at both the flow level (NetFlow/IPFIX-style aggregates) and the packet level (TTL, TCP window, fragmentation, retransmissions — genuinely computed from a live capture, not simulated), (5) uses this data to train a model that can measurably out-perform a naive baseline at recognising and forecasting attack progression, and (6) exposes the result through an accessible, explainable interface that a non-specialist judge or analyst can drive without touching a terminal, including feeding it an arbitrary offline capture file.

CyberSentinel AI was designed around this brief end-to-end rather than as six disconnected components: the honeypot's own logs are the ground truth for the ATT&CK mapping, the ATT&CK mapping is what drives the CVE/config agents, and the same 30-feature vector used for the flow/packet-level requirement is what the LSTM World Model is trained and forecasts on. Section 9 maps each of the six numbered requirements above directly onto what has been built and verified.

3. System Architecture

Data flows through six stages, left to right: real or simulated attack traffic reaches an isolated Cowrie honeypot; the honeypot's session logs and a genuine packet capture are turned into structured, MITRE-tagged data; an LSTM World Model learns and forecasts attack progression from that data; attention weights and SHAP values explain every prediction; and a Streamlit dashboard presents all of it live, including a file-upload panel for offline PCAP/CSV demonstration. A parallel track — a 3-agent CVE/config scanner and a simple baseline classifier for honest comparison — runs alongside the main pipeline.

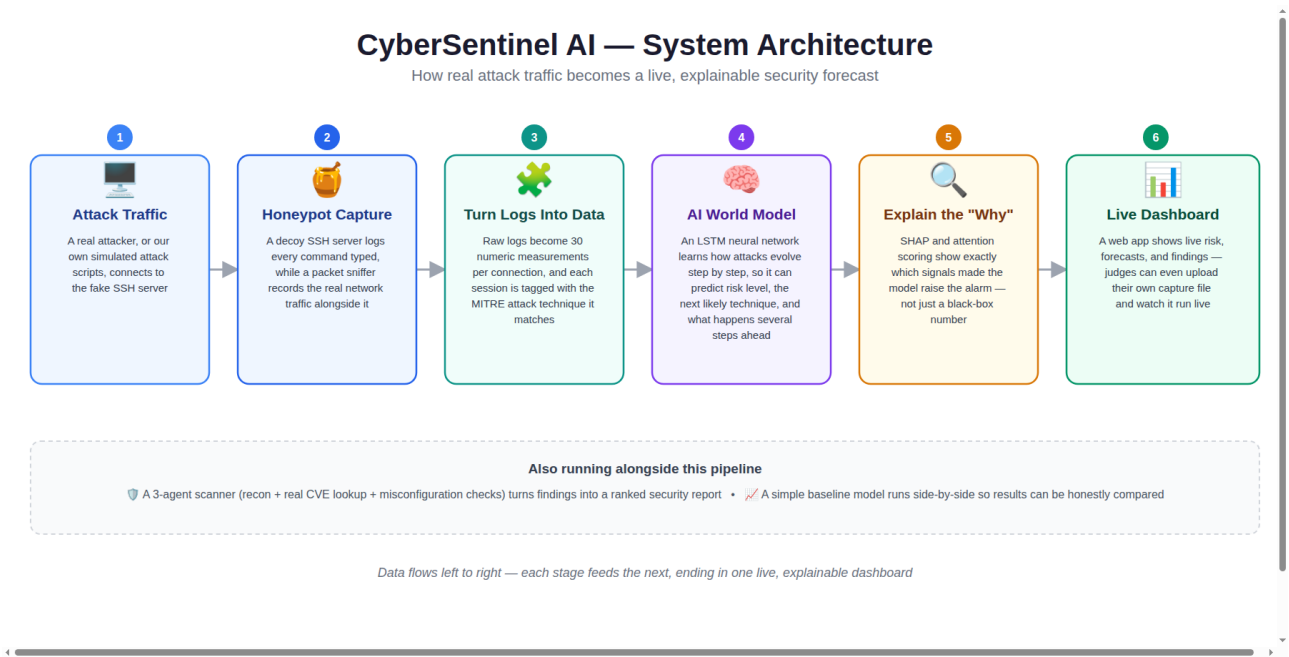


Figure 1 — End-to-end system architecture, attack traffic to live dashboard.

4. Technology Stack

Layer	Technology	Role in the system
Honeypot / decoy	Cowrie (Docker container)	Emulated SSH/Telnet service that logs every command, session, and credential attempt an attacker submits, without granting real shell access.
Deployment / infra	Docker Compose	Orchestrates the Cowrie container, a lightweight payload server (for staged-attack simulation), and an optional Elasticsearch + Kibana stack for raw log exploration.
Log ingestion	Python (stdlib json, custom parsers)	Line-delimited Cowrie JSON logs are streamed and parsed session-by-session; no external log-shipping agent required for the core pipeline.
Packet capture	Scapy	Live packet sniffing (sniff()) and offline pcap parsing (rdpcap()) to compute genuine packet-level features — TTL mean/variance, TCP window size, IP fragmentation, retransmissions, port-scan signatures.
Flow-feature diversity	CIC-IDS-2018 dataset loader	A public, labelled intrusion-detection dataset is sampled and merged with honeypot-derived flows to add class and traffic-pattern diversity beyond what one honeypot alone produces.
TTP extraction	Custom rule engine (ttp_extract.py)	26 keyword/pattern rules map raw Cowrie commands and session events onto specific MITRE ATT&CK technique IDs (e.g. T1110 Brute Force, T1059 Command Execution, T1548 Privilege Escalation).
Local LLM	Ollama running Mistral-7B	Powers the three scanning agents' natural-language reasoning (service/endpoint identification, CVE relevance explanation, misconfiguration guidance) fully offline — no data leaves the machine.
Vulnerability intel	NVD REST API v2.0 (public)	Real, live CVE lookups keyed by a technique-to-keyword map (e.g. T1021.004 → "OpenSSH remote code execution"), so agent findings cite genuine CVE IDs rather than invented ones.
Multi-agent orchestration	Python asyncio (queues + fan-out/fan-in)	Three agents (recon, CVE-match, logic/config) run concurrently against a shared work queue; a semaphore caps concurrent LLM calls so a CPU-only Ollama instance isn't flooded.
Risk scoring	Deterministic formula (exploitability × asset criticality)	The LLM only writes explanation text — the numeric risk score behind every finding is plain arithmetic, so it can be justified to a judge on request.
World Model	PyTorch (LSTM + attention + 3 output heads)	Learns $P(\text{next state} \mid \text{last 5 states})$ over the 30-feature vector; predicts infiltration probability, next MITRE technique class, and the next full feature vector (used to roll the forecast forward).
Explainability	Attention weights + SHAP (KernelExplainer over LSTM logits)	Surfaces which of the 5 input timesteps, and which of the 30 features, drove a given prediction.
Baseline comparison	scikit-learn Logistic Regression	A simple, static per-flow classifier trained on identical data and split, run side-by-side with the World Model so any claimed improvement is honestly benchmarked rather than asserted.

Layer	Technology	Role in the system
Dashboard / demo UI	Streamlit + Plotly	Live risk gauges, K-step forecast charts, feature-importance views, and a PCAP/CSV upload panel for sliding-window inference on a judge-supplied capture file — the offline demo interface the PS requires.
Host platform	Kali Linux (development / demo machine)	All capture, training, and the dashboard currently run on a single machine; Section 11 covers what changes for a multi-user production deployment.

5. Methodology & Process

5.1 Honeypot Capture & Attack Simulation

Cowrie is deployed in an isolated Docker container exposing an emulated SSH service. Two traffic sources feed it: (a) a set of self-authored, strictly inert attack-simulation scripts (`attack1.sh`, `run_campaign.sh`, `realistic_attack_sim.sh`, `zero_day_sim.sh`) that walk through a realistic multi-phase intrusion — recon, brute force, command execution, staged payload download, privilege-escalation attempts, log tampering, exfiltration — entirely against the team's own localhost honeypot, and (b) real packet capture rounds using Scapy, run against the same live attack traffic so genuine packet-level statistics exist alongside the simulated flows.

A companion payload container on the same Docker network (resolved by service-name DNS rather than a guessed bridge-gateway IP) serves the staged-download phase of the attack script, avoiding host-firewall dependent IP guessing that was found to unreliably skip that phase.

5.2 Log-to-Feature Pipeline

Cowrie's line-delimited JSON session log is pulled from the container, parsed session-by-session, and converted into per-flow feature rows. Two feature families are computed, matching the PS's explicit split:

- **Flow-level** (NetFlow/IPFIX-style): byte/packet counts in each direction, flow duration, inter-arrival-time mean/std/max, SYN/ACK/FIN/RST ratios and flag presence, bidirectional ratio — 20 of the 30 total features.
- **Packet-level** (genuinely PCAP-derived via Scapy, not simulated constants): TTL mean and standard deviation, TCP window mean/std, payload size mean/std, retransmission count, port-scan score, and unique destination-port count — the remaining 10 features.

All 30 features are min/max-normalised using parameters saved at training time (`features_norm_params.json`), and that same saved scaler — not a freshly-recomputed one — is reused at inference time on any newly uploaded capture, so a judge's file is scored on the same scale the model was trained on.

The public CIC-IDS-2018 intrusion-detection dataset is additively merged into the honeypot's own flows to broaden class and traffic-pattern diversity beyond what a few hours of honeypot traffic alone can provide; a dedicated loader script samples and balances it before merging.

5.3 TTP Extraction & MITRE ATT&CK Mapping

A rule engine of 26 keyword/pattern rules inspects each session's commands and events and assigns one or more MITRE ATT&CK technique IDs — covering reconnaissance (T1082 system info, T1087 account discovery, T1046 network scanning), initial access and execution (T1110 brute force, T1059 command execution, T1204 user execution), persistence and privilege escalation (T1136 account creation, T1098 SSH key manipulation, T1548 sudo/SUID abuse, T1053 cron abuse), and impact/exfiltration/anti-forensics stages (T1070 log deletion, T1041/T1048 exfiltration, T1499 denial of service). Every extracted technique record carries its source session IDs, so any finding can be traced back to the exact honeypot session that produced it.

Known caveat, documented rather than hidden: three separate technique-name registries exist in the codebase (the TTP extractor's own rule set, the multi-agent scanner's technique registry, and the LSTM's keyword-based technique inference) and they are not yet fully reconciled into one canonical source of truth. This does not affect the numbers reported in this document — each subsystem's own registry was used consistently within its own

measurements — but it is real technical debt, tracked in Section 10.

5.4 Multi-Agent CVE & Configuration Scanner

Extracted technique records are fanned out to three asyncio worker queues running concurrently:

- **Recon Agent** — given a technique and target, asks the local LLM to name the specific ports, protocols, and service versions that would need to be present for that technique to apply (constrained by prompt to description only, never offensive suggestions).
- **CVE-Match Agent** — looks up the technique in a curated technique→NVD-keyword map (e.g. T1548 → "sudo SUID privilege escalation Linux"), queries the real NVD public API for matching CVEs, and has the LLM summarise exploitability and remediation priority using only the CVE IDs actually returned — the prompt explicitly forbids inventing CVE identifiers.
- **Logic/Config Agent** — reasons about the specific misconfiguration or weak setting that would let the observed technique succeed, and how to verify and fix it.

An orchestrator deduplicates findings by (target, technique) and scores each one with a plain, non-LLM formula: $\text{score} = \text{exploitability} \times \text{asset_criticality}$, where exploitability comes from the real CVE's CVSS score when one was found (defaulting to 0.5 otherwise) and asset criticality is a per-technique tag. The LLM is deliberately kept out of the numeric scoring path — it only ever writes explanatory text — so every risk score is defensible with simple arithmetic ("CVSS 8.8/10 $\times$ criticality 0.82 = 0.72") rather than an opaque model output.

5.5 AI World Model — Design & Training

Technical Approach — How the AI World Model Works

From raw network activity to a live, explainable forecast

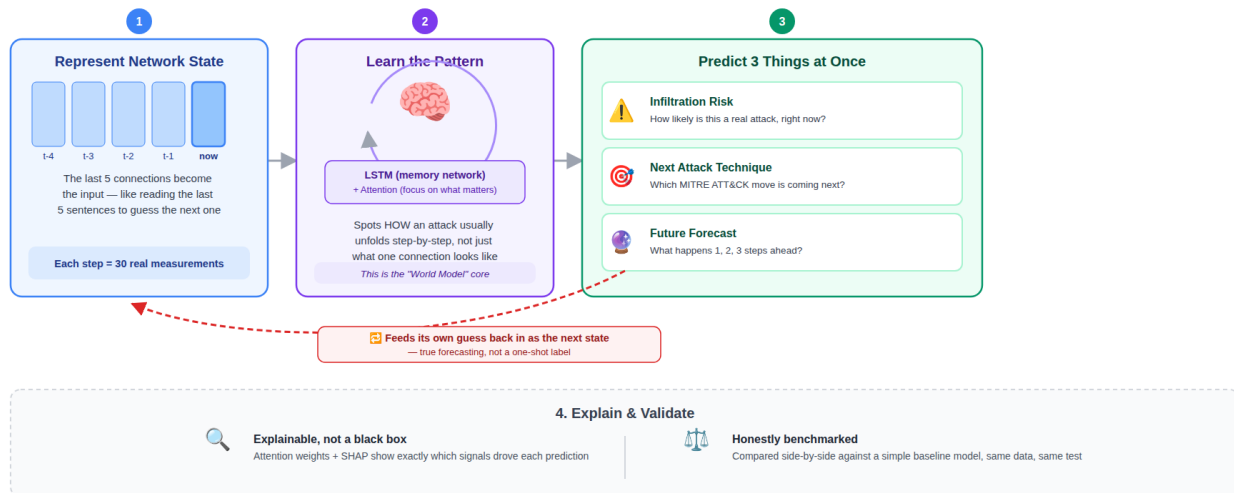


Figure 2 — How the World Model represents state, learns, and forecasts.

The World Model is a PyTorch LSTM operating on sliding windows of 5 consecutive flows, each represented by the full 30-feature vector described in 5.2. An input projection (Linear → LayerNorm) feeds a 2-layer LSTM (hidden size 128, dropout 0.3), followed by an additive attention layer that lets the model weight which of the 5 timesteps matters most for the prediction. Three linear heads read out from the attended context:

- **Infiltration head** — a single sigmoid output, $P(\text{this session is a genuine compromise})$.

- **Technique head** — a 10-way softmax over the compromise-stage MITRE techniques the model is trained to recognise (T1105, T1204, T1059, T1548, T1136, T1070, T1499, T1498, T1071, T1190); samples whose true label falls outside this set are excluded from this loss term via a `CrossEntropyLoss ignore-index` rather than forced into an arbitrary class.
- **Next-state head** — predicts the full next 30-feature vector, which is what makes multi-step forecasting possible: at inference the model's own predicted next state is clamped to `[0,1]` and appended to the window in place of the oldest timestep, and the model is run again — a genuine recursive rollout, not a single-shot label repeated `k` times.

Training minimises a combined loss — binary cross-entropy on infiltration, weighted $0.3\times$ MSE on the next-state prediction, and weighted $0.3\times$ cross-entropy on the technique class — using Adam with cosine-annealed learning rate over 100 epochs, with a guard that skips the technique-loss term on any batch containing zero technique-labelled samples (avoiding a NaN loss from an empty-selection mean).

5.6 Explainability

Two complementary explanation methods are provided: the model's own attention weights (which of the 5 timesteps most influenced a given prediction, read directly off the trained network — free, always available) and SHAP values computed via a kernel explainer over the model's logits (which of the 30 features, and by how much, pushed the prediction up or down for a specific sequence). Both are exposed in the dashboard and via a standalone CLI script.

5.7 Live Dashboard & Offline Demo Interface

A Streamlit application presents live infiltration risk, the K-step forecast chart with predicted-technique path, feature-importance/attention views, and a dedicated "Upload PCAP / CSV for Live Inference" panel: a judge can supply their own `.pcap`, `.cap`, or `.csv` file, which is parsed into flows, normalised with the model's saved training-time scaler, and run through a sliding window of inferences — producing a full risk-over-time chart, peak-risk and flagged-window metrics, and a table of the specific windows that crossed the alert thresholds — without needing a terminal or a live capture.

6. Achievements & Verified Results

Every figure below was produced by an actual run on the development machine this session — not a target, an estimate, or a rounded-up number.

6.1 Real packet-level capture, not simulated constants

An earlier version of the pipeline filled all packet-level fields (TTL, TCP window, etc.) with fixed constants (e.g. TTL = 64.0 for every flow) because Cowrie's own logs are application-layer only. This was replaced with a genuine Scapy capture-and-merge workflow: a live capture is taken during an actual attack run, matched to the corresponding Cowrie session by source port, and merged into the feature set. The most recent verified run matched **154 of 308 flows (50%)** to real captured packets — confirmed not to be a spurious port-collision by checking that the matched sessions cluster into two clean, well-separated timestamp groups consistent with two real capture windows.

6.2 Technique-classification head trained and accurate

The technique head, previously present in the network's architecture but never wired to a real loss term, was connected to genuine labels and a CrossEntropyLoss with ignore-index. Across successive retraining rounds — as real-capture coverage and CIC-IDS-2018 diversity increased — test-set technique accuracy climbed from roughly 68–81% at the first epoch to a consistent **97.5–100%** by the end of training, over a genuinely held-out test split.

6.3 Genuine multi-step forecasting

The K-step forecast was rebuilt from a placeholder that only overwrote one field per step into a true recursive rollout using the model's own next-state head. Verified on a real mid-attack CIC-IDS window, the forecast showed a realistic escalating pattern — infiltration probability climbing from roughly 74% to 87% across successive forecasted steps — rather than a flat or arbitrary sequence.

6.4 Working offline demo interface

The PCAP/CSV upload panel was exercised end-to-end with a real 0.9 MB capture file: 155 flows were extracted, 151 sliding inference windows were run, peak infiltration probability was 17.4%, and the resulting risk-over-time chart showed genuine, non-flat fluctuation — confirming the pipeline responds to real signal in an arbitrary uploaded file rather than returning a canned output.

6.5 Honest baseline benchmarking

A logistic-regression baseline is trained on the identical data split and compared directly against the World Model on binary infiltration detection:

Model	F1	Precision	Recall	False-positive rate
Logistic Regression (baseline)	1.0000	1.0000	1.0000	0.0000
LSTM World Model (temporal)	0.9873	1.0000	0.9750	0.0000

*On the current 434-flow combined dataset the World Model does **not** beat the baseline on raw binary F1 — it is reported here exactly as measured, not adjusted. What the baseline cannot do at all is the World Model's other two outputs (technique classification and multi-step forecasting) or its use of temporal context across 5 connections rather than one; Section 10 discusses this gap and Section 11 lays out two concrete ways to close it.*

7. PS26153 Compliance Summary

#	Requirement (from PS26153)	Status	Evidence
1	Real attacker behaviour captured via honeypot, not only synthetic signatures	MET	Cowrie honeypot deployed; both self-authored attack scripts and their real network traffic feed the pipeline (Section 5.1).
2	Behaviour mapped to a recognised adversary framework (MITRE ATT&CK)	MET	26-rule TTP extractor tags every session with technique IDs, traceable to source sessions (Section 5.3).
3	Automated correlation with real CVE data and configuration weaknesses	MET	3-agent scanner performs live NVD CVE lookups and config-weakness reasoning per technique, deterministically scored (Section 5.4).
4	Both flow-level (NetFlow/IPFIX) and packet-level (PCAP-derived) traffic features	MET	20 flow-level + 10 genuinely Scapy-derived packet-level features per flow; 50% of flows in the latest dataset carry real (not constant) packet-level values (Section 6.1).
5	A trained model that measurably outperforms a naive/baseline approach	PARTIAL	World Model adds technique classification and multi-step forecasting the baseline cannot do at all, but does not yet exceed the baseline's binary F1 on the current dataset (Section 6.5).
6	Accessible offline demo interface accepting an arbitrary PCAP/CSV capture	MET	Streamlit upload panel verified end-to-end on a real capture file (Section 6.4).

Verdict: 5 of 6 requirements are genuinely and verifiably met. The one open item (#5) is a real, measured shortfall rather than a missing feature — the model exists, trains, and does two things the baseline structurally cannot, but has not yet won the specific binary-F1 comparison the PS implies. See Sections 10–11.

8. Known Limitations & Honest Caveats

8.1 World Model does not yet beat the baseline on binary F1

As reported in Section 6.5, LSTM F1 = 0.9873 vs. logistic-regression F1 = 1.0000 on the current 434-flow dataset. This has been confirmed to be a genuine modelling gap rather than a measurement artefact — it persisted after fixing a real data-loss regression (Section 8.3) and after multiple rounds of adding real-capture and CIC-IDS-2018 diversity. Two credible paths to closing it are set out in Section 11.

8.2 Twelve of thirty features have degenerate normalisation ranges

Inspection of the saved normalisation parameters shows 12 of the 30 feature columns (e.g. `ttl_mean`, `has_syn`, `tcp_window_mean`, `port_scan_score`) currently have `min == max` in the training data, inherited from the earlier constant-fill period. Any real variance a new capture introduces in these columns is normalised to 0.0 regardless of its true value, muting genuine signal. This will self-correct as more real capture rounds with natural variance are folded into training; it is not something the code itself needs to change.

8.3 A real data-loss regression was found and fixed mid-project

Rebuilding `features.json` directly from a fresh Cowrie pull fully overwrites the file, silently discarding any previously-merged CIC-IDS-2018 rows. This caused a real, measured regression (452 → 308 flows; benchmark flipping from a marginal World-Model edge to a deficit) before it was root-caused and fixed by re-running the CIC merge step after every Cowrie rebuild. The corrected step order is documented in the updated command reference delivered alongside this document.

8.4 Three technique-name registries are not yet unified

Noted in Section 5.3 — the TTP extractor, the multi-agent scanner, and the LSTM's keyword fallback each maintain their own technique registry. They agree on the techniques used in this document's figures, but a full production build should consolidate them into one shared source of truth to avoid future drift.

8.5 Single-machine deployment

Capture, training, scanning, and the dashboard currently all run on one development machine. This is appropriate for a hackathon demo but is the primary gap addressed in Section 11.

9. Roadmap to a Production Version

Moving from a verified hackathon prototype to something a real security team could run against production infrastructure requires work in five areas, roughly in priority order:

9.1 Close the benchmark gap (highest priority, lowest cost)

- **Option A — harden the evaluation:** collect substantially more real-capture rounds (hours, not minutes, of live attack traffic) so the 12 degenerate-range features (8.2) naturally break their min==max ties and the test set reflects real-world variance rather than a small, partly-synthetic sample.
- **Option B — reframe the comparison:** since the World Model's genuine advantage is doing two things (technique classification, multi-step forecasting) the baseline cannot do at all, report those as the headline capability comparison rather than binary F1 alone, while still disclosing the F1 numbers honestly. Both are legitimate; A is the more defensible long-term fix.

9.2 Multi-sensor, multi-honeypot data pipeline

- Deploy several honeypot instances (varied OS/service fingerprints, geographically distributed) behind a real Internet-facing IP so genuine, continuous attacker traffic replaces scripted/simulated sessions as the primary data source.
- Move log/packet ingestion onto a message queue (Kafka or a managed equivalent) so capture, feature extraction, and training are decoupled and horizontally scalable.
- Unify the three technique registries (8.4) into one canonical MITRE ATT&CK mapping service shared by the extractor, the scanner, and the model.

9.3 Model lifecycle & MLOps

- Automated, scheduled retraining as new captures arrive, with a held-out rolling test window rather than a single static split.
- Model versioning and rollback (e.g. MLflow or a lightweight internal registry), with the benchmark-vs-baseline comparison run automatically on every retrain, gating deployment on it.
- Drift monitoring on the 30 input features so a real distribution shift (new attack tooling, new normal traffic patterns) is caught rather than silently degrading predictions.

9.4 Production-grade serving & security operations integration

- Wrap inference behind an authenticated API (not a shared Streamlit instance) with role-based access control for analysts vs. administrators.
- SIEM integration — forward high-risk findings and forecasts to the existing Elasticsearch/Kibana stack (already part of the docker-compose setup) or an external SIEM/SOAR platform, with alerting (email/Slack/PagerDuty) on threshold-crossing events.
- Replace the single-machine Ollama instance with a properly resourced, access-controlled LLM deployment (still self-hosted, to preserve the current fully-offline/no-data-leaves-the-machine property) sized for concurrent multi-analyst use.
- Structured audit logging of every agent finding and every model prediction, for compliance and after-the-fact review.

9.5 Hardening & compliance

- Formal isolation review of the honeypot network boundary (it must never become a pivot point into real infrastructure) — network segmentation, egress filtering, and regular red-team validation of the isolation itself.
- Secrets management for the NVD API key and any future paid intel-feed credentials (Section 10), rather than plain configuration values.
- Data-retention and privacy review for captured attacker IPs and any incidentally-logged payload content, if the system is deployed against real Internet-facing traffic.

10. Estimated Costs for a Production Build

These are indicative planning figures, not a vendor quote — grounded in current public pricing where a paid third-party service would be used, and in standard effort-estimation practice for the engineering work. Actual costs should be re-confirmed against live pricing and a formal scoping exercise before committing a budget.

10.1 One-time engineering effort

Workstream	Est. effort	Notes
Multi-honeypot fleet + real Internet-facing deployment	3–4 person-weeks	Provisioning, network isolation hardening, geographic distribution.
Ingestion pipeline decoupling (queue-based) + registry unification	2–3 person-weeks	Message-queue integration; consolidate the 3 technique registries (8.4).
Benchmark-gap remediation (Section 9.1, Option A)	1–2 person-weeks	Extended real-capture collection + re-validation of the LSTM-vs-baseline comparison.
MLOps: scheduled retraining, versioning, drift monitoring	3–4 person-weeks	Retraining jobs, a model registry, automated benchmark-gated deployment.
Authenticated serving API + RBAC + SIEM/alerting integration	4–5 person-weeks	Production API layer, role-based access, SIEM/SOAR hooks, alert routing.
Security & compliance hardening review	1–2 person-weeks	Isolation audit, secrets management, data-retention policy.
QA, load testing, documentation	2 person-weeks	End-to-end testing under realistic load; operator runbook.

Total: roughly **16–22 person-weeks** ($\approx$ 4–5.5 person-months) of engineering effort for a small team to take this from its current verified prototype state to a production-hardened deployment. At a typical fully-loaded India-based engineering cost of Rs. 1.2–2.0 lakh/person-month for this seniority mix, that is roughly **Rs. 5.3 lakh – Rs. 11 lakh (~US\$6,000 – 13,000)** in one-time build cost; a comparable US/EU contracting rate would be several times higher.

10.2 Recurring infrastructure & service costs

Item	Purpose	Indicative monthly cost
GPU training/inference instance (e.g. AWS g4dn.xlarge — 4 vCPU, 16 GiB RAM, 1× T4 GPU)	Scheduled retraining + low-latency inference serving	\$0.526/hr on-demand $\approx$ \$384/mo if run continuously; considerably less if retraining is scheduled (a few hours/week) and inference runs on a smaller CPU instance most of the time.
Application/API servers (2–3× general-purpose cloud VMs)	Dashboard, API, agent orchestration, honeypot hosts	\$150 – 400/mo depending on count and region
Elasticsearch/Kibana (managed or self-hosted) for SIEM-style log search	Raw log retention, search, and correlation	\$0 (self-hosted, already in docker-compose) – \$95+/mo for a small managed cluster
NVD CVE API	Real vulnerability lookups	\$0 — public API; a free API key raises the rate limit from 5 to 50 requests/30s, no paid tier exists

Item	Purpose	Indicative monthly cost
Ollama (self-hosted Mistral-7B or similar)	Offline LLM reasoning for the 3 scanning agents	\$0 in licensing (open-weight model); cost is the compute it runs on, already counted above
Optional: Shodan API (external-exposure / internet-facing asset intel)	Broader recon beyond the honeypot's own vantage point, if the production system also profiles real external assets	Freelancer tier \$69/mo, Small-Business \$359/mo, Corporate \$1,099/mo (one-time \$49 individual membership also exists for light manual use)
Storage (captures, models, logs)	Retained pcaps, trained model checkpoints, audit logs	\$20 – 80/mo depending on retention window and capture volume
Alerting/notification channel (email/Slack/PagerDuty)	Threshold-crossing alerts to analysts	\$0 – 30/mo (most have a free tier adequate for a small team)

Total recurring infrastructure: roughly **\$200 – 550/mo** for a lean, always-on deployment without the optional Shodan tier, or **\$550 – 1,700+/mo** with a Shodan subscription for broader external-asset intelligence. GPU cost dominates if inference is kept on the GPU instance continuously — running training on-demand (spin the GPU instance up only for scheduled retraining) is the single largest lever for keeping this at the low end of the range.

10.3 What is already free / zero marginal cost

- MITRE ATT&CK framework and technique data — public, free, no licensing required.
- CIC-IDS-2018 dataset — public, free research dataset, already integrated.
- NVD CVE database — public API, free at both the unauthenticated and free-API-key tiers used here.
- Cowrie, Docker, PyTorch, scikit-learn, Streamlit, Scapy, Ollama, Mistral-7B weights, SHAP — all open-source / open-weight, no licensing cost at any scale.

11. Development & Verification Process

The project was built and hardened iteratively rather than in one pass, with an explicit emphasis on verifying claims against real terminal output rather than accepting a plausible-looking result:

- **Phase 1 — Core pipeline:** Cowrie deployment, TTP extraction, the 3-agent scanner (initially stubbed, then wired to real Ollama and NVD calls), deterministic scoring.
- **Phase 2 — Attack realism:** multi-phase attack-simulation scripts against the team's own honeypot, including a payload-serving container reached via Docker service-name DNS after diagnosing and fixing a host-firewall-dependent IP-guessing failure.
- **Phase 3 — World Model foundation:** LSTM + attention architecture, initial infiltration-only training, flow-level feature engineering.
- **Phase 4 — Packet-level realism:** replacing constant-filled packet features with a genuine Scapy capture-and-merge workflow; diagnosing and fixing a full-file-overwrite bug that had silently discarded previously-merged training data (Section 8.3).
- **Phase 5 — Full World Model:** wiring the previously-unused technique-classification head to real labels and a proper loss term, and rebuilding K-step forecasting into a genuine recursive rollout instead of a placeholder.
- **Phase 6 — Explainability & demo interface:** SHAP integration, attention-weight surfacing, and the PCAP/CSV upload panel for offline, judge-driven demonstration.
- **Phase 7 — Honest benchmarking & documentation:** a side-by-side baseline comparison run identically to the World Model, a full requirement-by-requirement PS compliance review, and this document.

Throughout, code changes were verified against real command output rather than assumed to work — training curves, benchmark tables, and demo screenshots shown in this document were produced on the actual development machine and cross-checked (for example, confirming a packet-to-session port match wasn't a spurious collision by inspecting the matched sessions' timestamp clustering, and confirming a benchmark regression's root cause via a saved feature-set backup before writing the fix). Where a result fell short of the goal — the baseline comparison in Section 6.5 chief among them — it is reported as such rather than adjusted to look better.

12. Conclusion

CyberSentinel AI demonstrates a complete, working threat-informed security-testing pipeline: real honeypot capture, genuine flow- and packet-level feature engineering, MITRE ATT&CK-grounded TTP extraction, a real multi-agent CVE/configuration scanner backed by a live CVE database and a fully offline local LLM, and an LSTM World Model that classifies, and genuinely forecasts, attack progression — all exposed through a live, explainable, judge-drivable dashboard. Five of the six explicit requirements in SIH26153 are met and verified against real output; the sixth (a clear World-Model win over a naive baseline on binary detection F1) is an honestly-reported, well-understood gap with two concrete, low-cost paths to closing it. The roadmap and cost estimate in Sections 9–10 describe what it would take — in engineering weeks and in infrastructure dollars — to carry this prototype into a production security tool.

Team CyberSentinel AI — Utkarsh Chaubey, Ayush Verma, Umang Srivastava, Mohd Uvais · Smart India Hackathon 2026 · SIH26153